

Building a casino platform you can sell

A system design for a multi-tenant platform that runs your own casino brand first, then licenses the same core to partner operators — owning the money and identity layer while licensing every game and every odd.

Prepared for Fernando 22 August 2026 Draft v1 — for discussion

01 What you are and aren't building

SoftSwiss and Soft2Bet are not software companies that happened to get licensed. They are licensing, content-contracting and payments companies that happen to ship software. The software is the smallest of the three problems.

You are **not** building slot games, live dealer studios, random number generators, sports odds, trading and risk management, or card acquiring. Every one of those is a separate multi-year business with its own certification regime, and every serious platform on the market licenses all of them.

What you **are** building is the thing none of those vendors will sell you: **the system of record for player identity and player money**, plus the commercial and operational surface wrapped around it. When an operator asks “whose platform is this?”, they mean whose wallet holds the balance and whose back office runs the brand. That is the asset.

Your licence route shapes the architecture

You've picked Anjouan. That is a defensible way to start — a single licence covers B2C and B2B, first-year cost lands near €33,000 against roughly €70–110k for Curaçao under the new LOK regime, and issuance is fast. But it comes with three constraints that are architectural, not just commercial:

- **Market blocking is mandatory and audited.** The licence prohibits serving the US, UK, France, Germany, Netherlands, Spain, Australia, Austria and FATF-blacklisted states. Geo-restriction has to be enforced in the platform, configurable per brand and per game, and logged — not left to a CDN rule someone can turn off.
- **Payments are the hard part, from day one.** Stripe, PayPal, Adyen, Worldpay and Checkout.com will not board Anjouan operators. Specialist high-risk acquirers charge roughly 8–12% against 2–3% for tier-1 licensees, and typically hold rolling reserves. EU banking is largely closed. This means payment orchestration and crypto rails are *phase one*, not a later refinement.
- **The licence is not what makes you sellable.** Partner operators will ask what your platform is *certified* to, not where it's licensed. GLI-19 certification, an uptime record and an integration count are what close B2B deals. Plan for certification early — see §8.

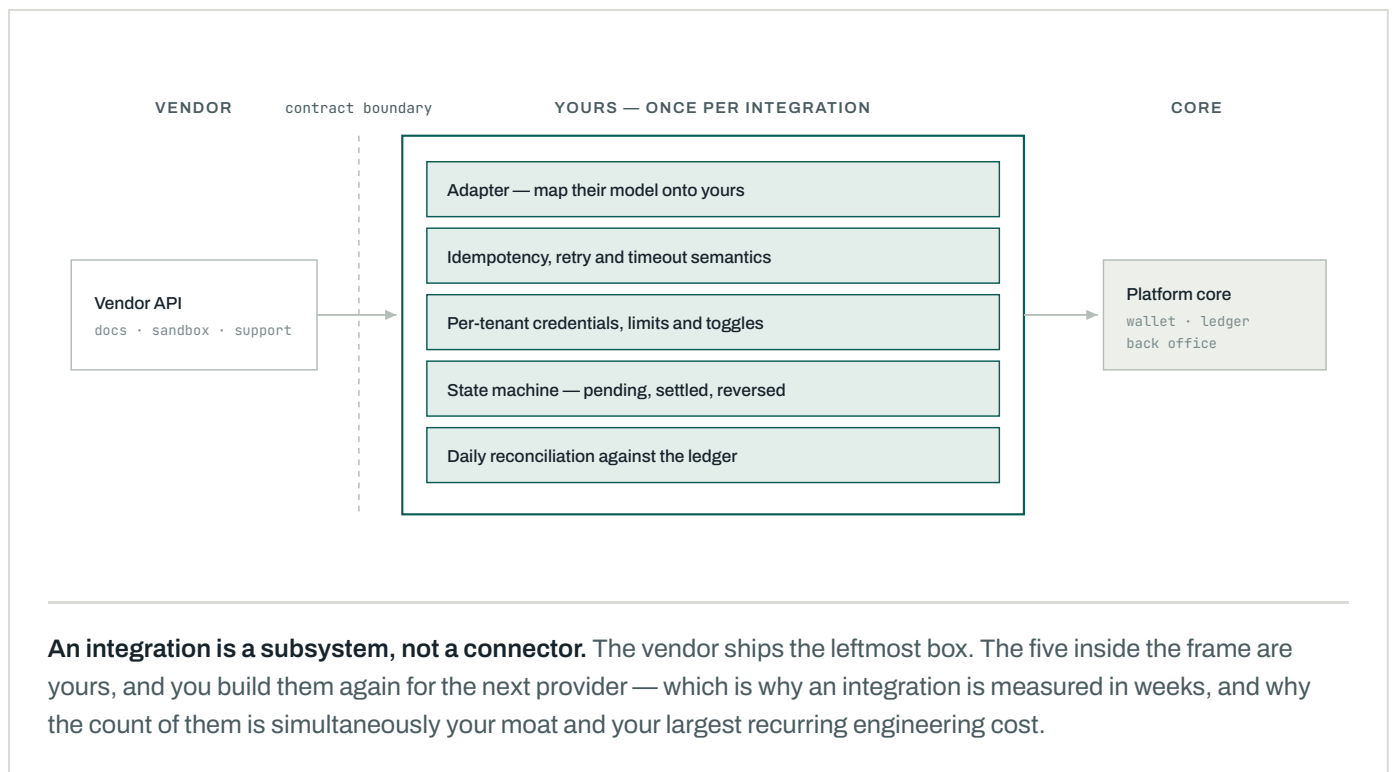
DESIGN CONSEQUENCE

Build jurisdiction as a first-class, pluggable concept from the first commit. The same core must be able to run an Anjouan brand and, later, a Curaçao or MGA brand with different KYC thresholds, responsible-gaming rules, reporting formats and data-residency requirements. Retrofitting that is a rewrite.

The build/buy line, component by component

Most of what a player touches is licensed. But “it's all integration” is the wrong mental model, for two reasons: several components that *look* like integrations carry a large platform-side subsystem the vendor does not supply, and the components with no vendor at all are precisely the ones a partner is buying from you.

COMPONENT		WHAT THE VENDOR GIVES YOU	WHAT YOU STILL HAVE TO BUILD
Casino games	BUY	Game content, RNG, certificates, the game UI itself	Catalogue sync, RTP-variant handling, jurisdiction blocklists, lobby and ordering, launch-token minting, free-round normalisation
Live casino	BUY	Studios, dealers, streaming, table UI	The above, plus table-limit configuration per brand and per currency, and dedicated-table commercials if you want branded tables
Sportsbook	BUY	Odds, trading, risk management, betting UI	Open-bet liability accounting, settle / void / partial / cashout as distinct ledger events, shared-wallet auth, GGR reporting that shows open liability
Cashier & payments	MIXED	Individual PSP APIs, acquiring, crypto rails	Orchestration and routing, cascade on decline, health-based failover, reserve accounting, the withdrawal approval workflow, and the cashier UI itself — one of the most-touched screens in the product
KYC / AML	MIXED	Document verification, liveness, sanctions and PEP lists	Tiered trigger rules, the case queue, per-jurisdiction thresholds, evidence retention, and actual enforcement — blocking play and withdrawal is your code, not theirs
CRM & campaigns	MIXED	Segmentation, journey builder, email / SMS / push delivery	A complete, clean event stream out, a segment API back in, and bonus-grant hooks — a campaign that can't issue a bonus is a newsletter
Affiliate tracking	BUY	Link tracking, commission plans, affiliate portal	Attribution capture at registration and accurate FTD / deposit / NGR postbacks. Third-party trackers cost real money and hold your data, so many platforms bring this in-house in year two
Bonus engine	BUILD	Nothing operators will accept off the shelf	All of it — rule engine, wagering and contribution, abuse controls — plus an adapter per provider, because every one exposes free rounds differently
Wallet & ledger	BUILD	—	All of it. Nobody sells you this, because it <i>is</i> the platform
Identity & PAM	BUILD	—	All of it, including cross-brand person resolution for self-exclusion and multi-accounting
Tenancy & config	BUILD	—	All of it. This is what turns every row above into a product instead of one casino
Back office & partner console	BUILD	—	All of it, and it is judged daily by people who are not your customers' engineers
Audit, RBAC, reporting	BUILD	—	All of it. First thing an auditor asks for, and the thing partners quietly compare you on



Two consequences follow. First, **integrations never finish**. Provider APIs change, games get added, currencies get enabled, and someone has to be on call when a provider's sandbox drifts from production. At steady state, budget roughly one engineer per fifteen to twenty live integrations for maintenance alone, before any new work.

Second, **your rates depend on volume you don't have yet**. Aggregators and acquirers quote against projected GGR, so a platform with no live brand gets the worst terms available. That is another argument for §1's recommendation below: your own brand generates the volume that makes the second contract cheaper than the first.

The one strategic recommendation

Run your own B2C brand first, on your own platform, with your own money at risk. Not as a detour from the B2B plan — as the only credible way to reach it. Every platform that sold well was proven on a live brand first. You will discover the rollback storms, the reconciliation drift and the bonus-abuse patterns on your own P&L rather than a partner's, and your first sales conversation starts with a running casino rather than a slide.

02 The product, in three surfaces

A B2B platform is sold on what people can see and operate. Three distinct surfaces sit on one core, and confusing them is a common early mistake.

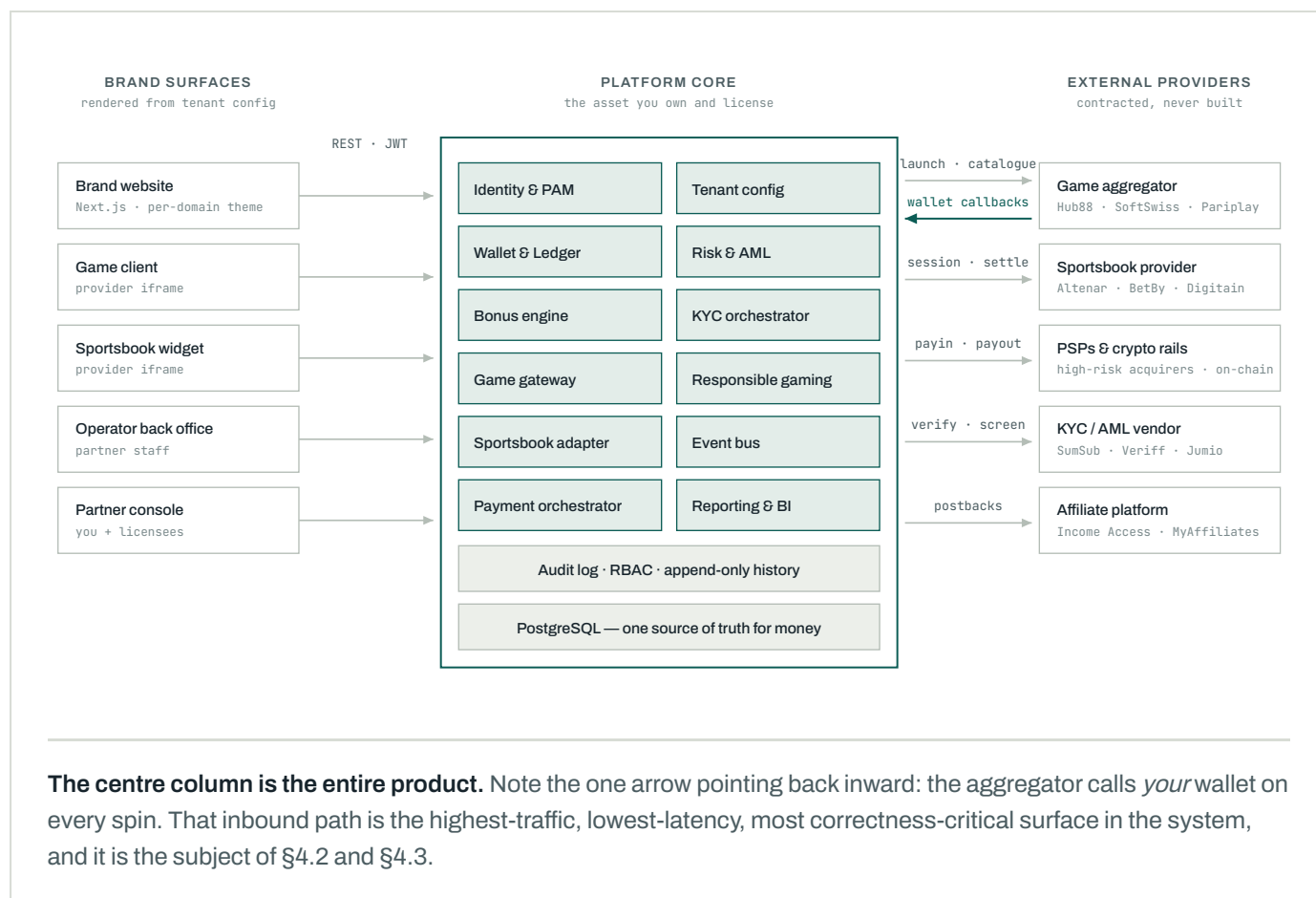
SURFACE	WHO USES IT	WHAT IT MUST DO
Brand front end	Players	Lobby, game launch, sportsbook, cashier, account, responsible-gaming controls. Generated from tenant configuration — theme, catalogue, languages, currencies, payment methods — never forked per brand.
Operator back office	Your partner's staff	Player management, bonus campaigns, payment approval, risk queues, CMS, reporting. This is the product they judge you on daily. It has to be usable by a non-technical retention manager.
Partner console	You and your licensees	Brand provisioning, provider and PSP credential management, revenue-share statements, cost pass-through, invoicing, platform-wide compliance view.

RULE OF THUMB

If launching a new brand requires an engineer, you have a consulting business. If it requires a form in the partner console and a DNS record, you have a platform. Everything in §5 exists to make the second sentence true.

03 System map

The centre column is the platform — the code you write, own and license. Everything on the left is rendered from tenant configuration. Everything on the right is a commercial contract with an API attached.



04 The core services

Nine services. For each, the decisions that are expensive to reverse.

4.1 Identity and player account management

Player identity is **per brand** — the same person can hold accounts at three of your partners' casinos and that is normal. But you also need a **cross-brand person key**, or you cannot enforce platform-level self-exclusion or detect multi-accounting, and both will be asked about in an audit.

Model it as one `player` row per `(tenant_id, email)`, linked to a `person` cluster resolved after KYC from a hash of document number plus date of birth. Self-exclusion, fraud links and AML case history attach to the `person`, not the `player`.

Session and game tokens are different things

The brand front end holds a short-lived JWT. Game launch mints a **separate, single-use, opaque token** bound to `(player, provider, game, currency, mode)` with a short TTL, which the provider exchanges once for a session. Never hand a provider your player session token — a leak at any one of thirty integrations otherwise becomes account takeover across your whole platform.

4.2 Wallet and ledger

This is the part that must be right on day one, because it is the part you can never retrofit.

Use **append-only double-entry accounting**. Never `UPDATE` a balance. Every movement of money writes balanced entries; the balance is a projection you can always recompute from the entries.

Account types

`player_cash`, `player_bonus`, `player_locked` (open sportsbook stakes), `house_gaming`, `provider_payable`, `psp_clearing`, `psp_reserve`, `jackpot_contribution`, `promo_liability`, `manual_adjustment`. Every entry carries `tenant_id`, `currency` and a transaction group id.

Money representation

Integers in minor units, with a per-currency exponent. If crypto is in scope — and given Anjouan it almost certainly is — `BIGINT` in cents is not enough: use `NUMERIC(38,0)` plus an exponent of 8 or 18 per asset. Floating point anywhere near a balance is a defect, not a shortcut.

Idempotency is the whole correctness story

A unique constraint on `(provider_id, provider_tx_id)` is what stops a retried callback from debiting a player twice. Rollbacks write compensating entries, never deletions — and a rollback for a transaction you have never seen must write a **tombstone**, so the original arriving late is rejected rather than applied.

BET — €10.00 stake, part bonus-funded

ACCOUNT	DEBIT	CREDIT
player_cash	4.00	
player_bonus	6.00	
house_gaming		10.00

$\Sigma \text{ debits} - \Sigma \text{ credits} = 0$

idempotency key: (pragmatic, tx_88f31c)

WIN — €25.00, same round

ACCOUNT	DEBIT	CREDIT
house_gaming	25.00	
player_bonus		25.00

$\Sigma \text{ debits} - \Sigma \text{ credits} = 0$

bonus-funded stake → win lands in bonus, not cash

balance(account) = Σ entries — materialised for reads, never authoritative

recomputed from the ledger hourly and diffed against the projection · any non-zero drift is a P1 incident

One bet, three entries, sum zero. Splitting the stake across cash and bonus at the ledger level — rather than tracking bonus in a side table — is what makes wagering progress, max-cashout caps and bonus-cost reporting fall out of the data instead of being reconstructed later.

WHERE PLATFORMS LOSE MONEY

Caching balances in Redis and reading them on the bet path. A cache read that races an in-flight write produces negative balances, free spins that never debit, and a reconciliation gap you will find weeks later in a partner's revenue report. The authoritative balance read happens inside the same database transaction as the write. Redis holds sessions and configuration, never money.

4.3 Game gateway and aggregator integration

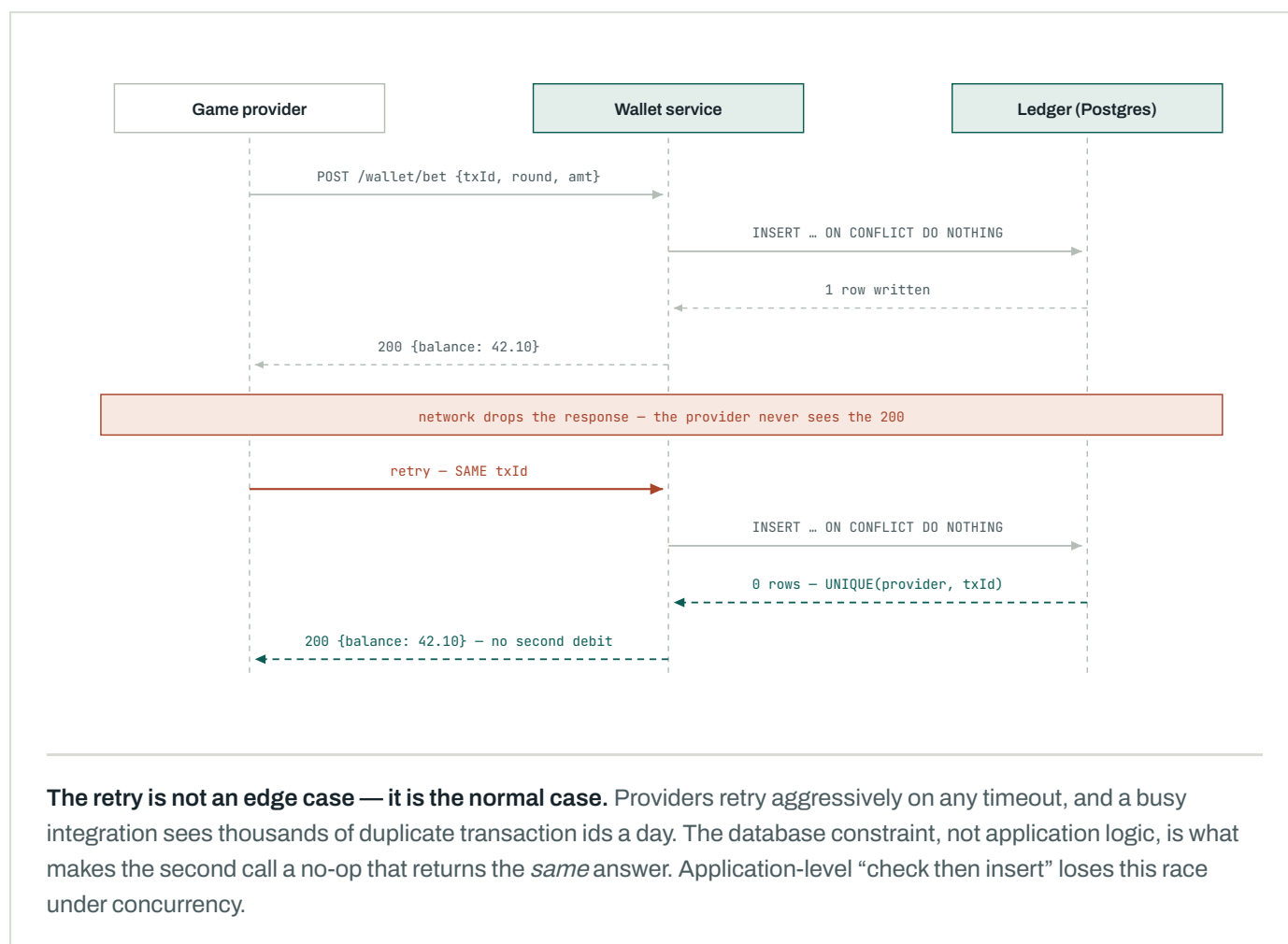
One integration with an aggregator gets you thousands of games. Write to an **internal provider interface** anyway — `launch()`, `catalogue()`, and inbound `bet` / `win` / `rollback` / `balance` — because you will end up with two or three aggregators plus direct integrations. Evolution and Pragmatic Play increasingly want direct commercial relationships, and premium content is what partners shop for.

Catalogue is not a list of names

Per game you need: provider game id, RTP variants (the same title ships at 96% and 88% — which one you get is contractual), volatility, feature flags, supported currencies, mobile support, demo support, and a **jurisdiction blocklist**. Some titles are illegal in some markets; serving one is a licence problem, not a bug. Sync nightly plus on webhook.

Free rounds are where the abstraction leaks

Every provider exposes free spins differently — some issue them provider-side against a campaign id, some expect you to credit them. This is the single messiest part of aggregator work and the reason the bonus engine (\$4.5) must talk to a normalised interface rather than provider APIs directly.



4.4 Sportsbook

You will not build odds, trading or risk management. Two integration shapes exist:

- **Widget / iframe** (Altenar, BetBy, Digitain) — the provider renders the whole betting experience; you supply authentication and a seamless wallet. Weeks of work.
- **Feed and API** — you receive odds and place bets programmatically, and build the UI. Months of work, more control over presentation, and you still don't own the risk.

Start with the widget, on the *same* wallet as casino so the player sees one balance. Then handle the part that genuinely is yours to model:

An open bet is a liability that spans days

A casino round settles in milliseconds. A sportsbook bet can sit open for months, settle in parts (bet-builder legs), be voided, be re-settled when a market is corrected, or be cashed out early at a price. Model the stake as moving into `player_locked` at placement, with an open-bet record carrying potential return. Settlement, void, partial settlement and cashout are each distinct ledger events.

REPORTING CONSEQUENCE

Sportsbook GGR is only known at settlement. A daily revenue report that ignores open liability overstates today and understates the day a big accumulator settles. Every sportsbook report needs an explicit open-liability line, or your partners will not trust your numbers — and they will be right not to.

4.5 Bonus and promotion engine

This is the component operators evaluate you on hardest, and the one most often built as an afterthought. Build it as an **event-driven rule engine** subscribed to the bus: `player.registered`, `deposit.settled`, `round.settled`, `bet.settled`, `session.started`.

The data model is four layers: **Campaign** (a marketing container) → **Offer** (the rules) → **Grant** (one player's instance) → **Progress** (an append-only trail of every wagering change). That progress trail is what you show a player who disputes a forfeited bonus, and there will be such players every week.

CONFIGURATION AXIS	WHAT OPERATORS EXPECT TO CONTROL, WITHOUT AN ENGINEER
Eligibility	Segment, country, currency, deposit method, min/max deposit, first-deposit-only, VIP tier, opt-in required
Reward	Match percentage and cap, fixed amount, free spins package (game, count, bet level), free bets, cashback percentage and window
Wagering	Multiplier, contribution percentage by game / category / provider, max bet while wagering, excluded games, time limit
Payout	Max cashout, cash-first or bonus-first bet ordering, forfeiture rules, partial release thresholds
Abuse controls	Low-risk betting detection, device and payment fingerprint linking, velocity caps, manual review queue

4.6 Payments and PSP orchestration

No PSP SDK ever appears in business logic. An orchestration layer routes on `(brand, country, currency, method, amount)`, cascades on decline, and fails over on provider health. With Anjouan economics, this layer is not a nicety — switching a declining acquirer quickly is the difference between a profitable month and a dead one.

Model the reserve explicitly

High-risk acquirers commonly hold a rolling reserve — frequently 5–10% held for 180 days. If that sits outside the ledger, your cash position is a fiction. Give it a `psp_reserve` account and release it on schedule.

Crypto: decide custody before you write code

Self-custody with HD wallets and a hot/cold split is cheaper and makes you a custodian of other people's money with all that implies. A custodian (Fireblocks, BitGo) costs real money and removes an entire category of existential risk. At year one, with a small team, the custodian is usually right. Either way you must handle per-asset confirmation thresholds, chain reorganisations, dust, memo/tag chains, and deposits sent on the wrong network — that last one happens weekly and needs a support workflow, not just an error log.

Withdrawals are a workflow, not an endpoint

Approval thresholds, KYC gating, velocity and pattern checks, a manual review queue, and four-eyes above a configurable amount. Never auto-pay above a threshold the operator sets.

KEEP OUT OF PCI SCOPE

Hosted fields or redirect only. Your servers never see a card number. The moment a PAN touches your infrastructure you inherit PCI-DSS obligations that will cost more than the feature is worth.

4.7 KYC, AML and responsible gaming

Treat these as a single compliance subsystem behind vendor-agnostic interfaces — partners will arrive with an existing SumSub or Veriff contract and expect you to accommodate it.

- **Tiered KYC** keyed to lifecycle events: registration, cumulative deposit thresholds, first withdrawal, enhanced due diligence above configurable limits.
- **Screening** against sanctions and PEP lists at registration and on a recurring schedule, not once.
- **Transaction monitoring** with configurable rules, a case management queue, and suspicious-activity report export.
- **Responsible gaming**: deposit, loss, wager and session limits — where a *decrease* takes effect immediately and an *increase* only after a cooling-off period. Reality checks, time-outs, self-exclusion at brand and platform level, and a permanent flag that survives account closure and re-registration.

None of these are features in the product sense. An audit is largely an examination of whether they are enforced *in the platform* and *logged immutably*. Build the logging with the control.

4.8 Back office, partner console and audit

Three-tier role-based access control: platform administrator (you), partner administrator, brand operator roles. Permissions are scoped by tenant and never inferred from the UI.

Every mutating action writes an audit record — actor, tenant, entity, before state, after state, IP, reason code — to an append-only store with 5–7 year retention. Manual balance adjustments require a reason code and four-eyes approval above a threshold. This is not defensive engineering; it is the first thing an auditor asks to see, and its absence has ended platform businesses.

4.9 Reporting and BI

Never run reports against the transactional ledger. Change data capture (Debezium) feeds the event bus, which feeds ClickHouse. The operational database keeps its latency budget; analysts get columnar speed.

The reports that get looked at daily: GGR and NGR sliced by brand, game, provider, country, currency and day; bonus cost; provider cost; PSP cost; first-time depositors; retention cohorts; player lifetime value. Regulatory exports are per-jurisdiction and belong behind a pluggable interface for the same reason \$1 gave.

05 Multi-tenancy: the decision that makes it a product

Everything brand-specific must be data. The moment one brand needs a code path, you have started a consulting business with extra steps.

Configuration domains

Theme and CMS content, game catalogue and lobby ordering, payment methods and limits, currencies, languages, responsible-gaming defaults, bonus templates, jurisdiction rules, provider credentials, domains and SEO. All rows in a database, all editable from the partner console, all versioned.

Isolation, and a path to more of it

Start with a shared PostgreSQL cluster, `tenant_id` on every row, enforced by **row-level security** bound to a connection-level tenant setting. Not by remembering to write `WHERE tenant_id = ?` — that discipline fails exactly once, and once is enough.

Then design the data access layer so tightening isolation is a deployment concern rather than a rewrite: schema-per-tenant, then database-per-tenant, then cluster-per-tenant. A regulated partner or a tier-1 jurisdiction will eventually demand one of these, and the ability to say yes without a migration project is worth real money.

ASK EVERY AGGREGATOR THIS IN CONTRACT TALKS

Does your integration permit sub-operators under our credentials, or does each brand need its own contract and integration? The answer determines whether partners inherit your content deal or bring their own — a commercial question with a direct schema consequence for how you store provider credentials and attribute provider cost. Get it answered before you design the game gateway, not after.

06 The numbers that actually bind

Non-functional requirements for a gaming platform are not aspirations. Providers monitor your integration and suspend it when you miss; partners read your uptime in their revenue reports.

PROPERTY	TARGET	WHY THIS NUMBER
Wallet callback p99	< 150 ms	Providers enforce timeouts and measure you. Slow responses trigger retries, then rollback storms, then suspension.
Platform availability	≥ 99.95%	Roughly 4 hours a year. Aggregator contracts routinely set availability floors below which they can disable the integration.
Ledger RP0	0	Synchronous replica. Losing committed financial transactions is unrecoverable in a way that losing a session is not.
Ledger RT0	< 15 min	Automated failover, rehearsed. A manual runbook at 3am is not a plan.
Reconciliation drift	0	Projection versus recomputed ledger, checked hourly. Any non-zero result pages someone.
Game launch p95	< 800 ms	Directly measurable in player drop-off between lobby click and first spin.
Audit retention	5–7 years	Immutable, exportable. The typical regulatory floor; assume the longer figure.
Report freshness	< 5 min	Retention teams act intraday. Yesterday's numbers are a compliance artefact, not an operating tool.

07 Technology choices

LAYER	CHOICE	REASONING
System of record	PostgreSQL 16+	Real transactions, serialisable isolation where it matters, partitioning, row-level security. Money needs strong consistency; there is no eventual-consistency story for a balance.
Wallet service	Go	Predictable tail latency without GC pauses. This is the one service where p99 is a contractual number.
Other services	Go or Kotlin	Node/TypeScript is fine for back-office APIs. Keep one language for the hot path and one for everything else; three is a hiring problem.
Event bus	Kafka	Per-player ordering matters for the bonus engine. NATS JetStream is a lighter alternative if ops capacity is the binding constraint.
Analytics	ClickHouse	Fed by CDC. Keeps reporting load entirely off the ledger.
Cache and sessions	Redis	Sessions, tenant config, rate limits. Never balances (§4.2).
Brand front end	Next.js	Server rendering matters — casino affiliate traffic is SEO-driven and a client-rendered lobby costs you acquisition.
Back office	React + a real data grid	Operators live in tables of tens of thousands of rows. Virtualisation and server-side filtering are requirements, not polish.
Hosting	VERIFY FIRST Hetzner, OVH, Leaseweb	AWS and GCP have acceptable-use policies covering gambling that vary by region and licence. Confirm in writing that your provider permits your activity <i>before</i> you build on it — migrating a certified platform is painful.
Secrets	Vault or cloud KMS	Provider HMAC keys rotate; PSP credentials are per-tenant. Config files are not an option.
Observability	OpenTelemetry	Per-provider latency SLOs and per-tenant error budgets. You need to know which integration is slow, not that something is.

08 Certification, and what it does to your engineering

GLI-19 (Interactive Gaming Systems) is the standard that covers a platform of this shape; **GLI-33** (Event Wagering Systems) covers the sportsbook side. Random number generator certification lives inside GLI-19, but you are not shipping RNGs — your content providers are, and their certificates are part of your evidence pack rather than your obligation.

In engineering terms, certification means: reproducible builds, pinned versions, documented and enforced change control, a release process someone external can follow, and the discipline that certified components cannot change without notification or recertification. Budget three to six months and low-to-mid five figures per scope, and

start the conversation before you have written much — it shapes how you release, and retrofitting release discipline onto a team that ships freely is harder than adopting it early.

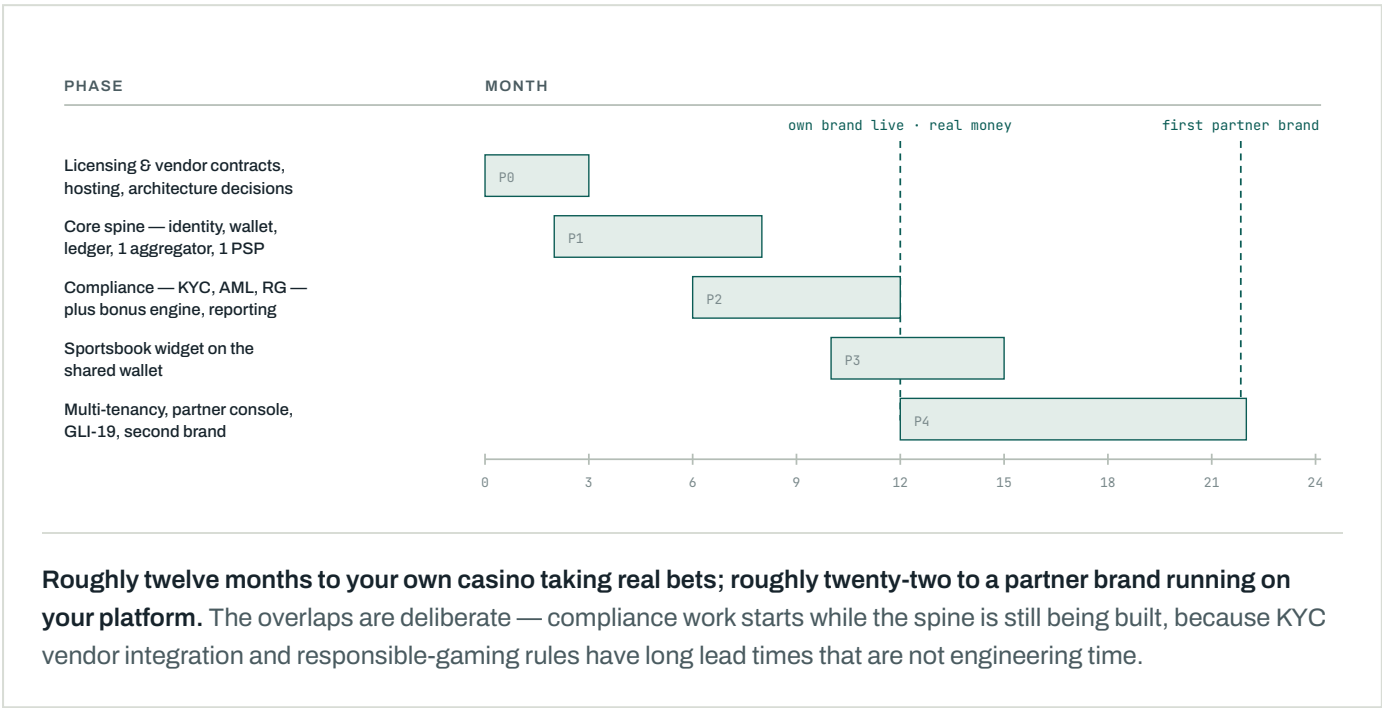
Serious partners will also ask for ISO 27001. Treat it as a year-two item, but let the security decisions you make now be compatible with it.

THE COMMERCIAL POINT

For a B2B platform, certification is worth more than the licence. An Anjouan licence with GLI-19 certification opens conversations that an Anjouan licence alone does not, because the partner's own regulator asks what the platform is certified to.

09 Build sequence

The ordering below is driven by dependency, not preference. The wallet and ledger gate everything; compliance gates going live with real money; multi-tenancy hardening only pays off once there is something proven to make multi-tenant.



What it takes to hold that timeline

Four to six backend engineers, two front end, one SRE, one QA with a payments and gaming background, one product owner. Alongside them, and not optional: a compliance officer, and someone whose actual job is provider and PSP relationships — contract negotiation with aggregators and acquirers is a full-time role that founders routinely underestimate.

Fully loaded in Malta, that team runs roughly €1.2–2m a year. On top sit vendor revenue shares — aggregators commonly take 8–15% of casino GGR, sportsbook providers 8–12% — plus PSP costs at Anjouan rates, licensing and certification.

HONEST READ

A smaller team can absolutely build this, but it takes longer, not less — the timeline stretches roughly in proportion. What does not compress at any team size is vendor contracting, licensing and certification, which run on other people's calendars. Start those in month zero.

10 Decisions that block the design

Six answers change the architecture materially. Until they exist, any schema written is provisional.

Q1 Crypto-first or fiat-first?

Changes ledger numeric precision, the entire payment layer, KYC thresholds and your PSP shortlist. Given Anjouan's banking reality this leans crypto-first, but it should be a decision rather than a default.

Q2 Do partners sit under your licence, or bring their own?

Determines aggregator contract shape, whether provider credentials are per-platform or per-tenant, how provider cost is attributed, and how strictly tenants must be isolated. This is the single highest-leverage commercial question.

Q3 Which markets in year one?

Drives languages, currencies, PSP selection, KYC vendor coverage, game jurisdiction blocklists and responsible-gaming defaults. “Everywhere that isn't blocked” is not an answer a payment provider will accept.

Q4 Self-custody crypto, or a custodian?

A security and insurance decision more than a technical one. At year one with a small team, a custodian is usually the right call despite the cost.

Q5 Is your own B2C brand part of the plan?

§1 argues strongly that it should be. If it is not, you need a design partner willing to launch on an unproven platform, and you will pay for that in equity or margin.

Q6 Which host has confirmed in writing that it permits your activity?

Cheap to check now, expensive to discover after certification. Get it in the contract.

Sources for figures cited

1. Anjouan licence costs, restricted countries and payment-processing constraints — [Gambling Law Index, Anjouan licence restrictions 2026](#)
2. Curaçao LOK fees, substance requirements and timeline — [Point Legal, Curaçao gaming licence cost 2026](#)
3. MGA licence categories, fees and compliance contributions — [Gaming Compliance, MGA licence requirements](#)
4. GLI-19 and GLI-33 scope and regulator acceptance — [Gaming Compliance, GLI standards](#)

Fee figures, vendor terms and revenue-share ranges are indicative and move; verify each against a current quote before committing to a plan. Nothing here is legal advice — licensing structure, corporate substance and tax treatment need a gaming lawyer in the relevant jurisdiction, and Malta has several who do this daily.